Part A - My first 90 days

Period	Main action	Why
Days 1-14	Understand the failures, assign incident owners, confirm launch scope and dependencies	Establish control before making large changes.
Days 15-42	Fix the highest-risk money gaps and test region-4 payment journeys	Improve safety while keeping launch work moving.
Days 43-70	Rehearse recovery, prove readiness, pilot and launch	Find problems before exposing the full customer base.
Days 71-90	Stabilise, measure results, and demonstrate backup coverage	Confirm the improvements work without relying on S. or T.

Diagnosis and working assumptions

My initial diagnosis is that the organisation depends on individual expertise where it needs repeatable safeguards. Money defects passed review, so another approval alone will not fix the problem. S. is a wallet knowledge bottleneck; T.'s absences expose the same weakness in reporting. Late QA discoveries, rising recovery times, and an unexamined launch overrun indicate weak feedback from production into delivery planning.

I would validate this diagnosis in week one through engineer/QA/Support conversations, incident timelines, representative payment changes, and the region-3 launch history. The predecessor's opinions are hypotheses, not performance findings.

Assumptions to validate early:

- **Decision-makers are available:** Product, Finance, and Compliance can identify people authorised to approve region-4 scope, financial controls, and regulatory readiness.
- **Optional features can wait:** Some launch features can be postponed to meet the deadline. Money safety and legal requirements must still be met.
- **Only confirmed staff are allocated:** The scenario says 20 engineers, but the squads total **18 (7 + 5 + 6)**. Use those 18 until the remaining two engineers' availability is clarified.
- **Hiring is not a launch dependency:** Assume the existing team can deliver the agreed launch scope. The two vacancies may help later, but the launch cannot depend on filling them in time.

Priorities, sequence, and trade-offs

Days 1-14: establish control and a credible launch plan.

- Name an incident lead, technical responder, and communications owner for each serious incident; publish an escalation route and primary/backup rota. Confirm access, alerts, and safe ways to suspend affected money flows. Review recent money incidents immediately and hold a focused region-3 retrospective with owners and dated actions.
- Map deposit, wallet, withdrawal, and reconciliation paths. D. and another engineer work with S. to find where duplicate protection, atomic balance/ledger posting, decimal handling, and bounded batch execution already exist or are missing. Fix the highest-exposure gaps first; do not begin a wallet rewrite.
- By day 10, agree region-4 minimum scope, named owners, provider dependencies, and regulatory acceptance criteria with Product/Compliance. By day 14, require provider contacts, sandbox access, credential ownership, and a milestone forecast. Unresolved external dependencies become an immediate CEO decision, not a surprise in week 9.

Days 15-42: make the critical path safe and observable.

- Implement or reuse one wallet-posting path with database-enforced duplicate protection, atomic balance/ledger updates, decimal arithmetic, and brand isolation. Exercise retries, concurrent writes, and failures against the database. Run reconciliation for live money flows with Finance; investigate discrepancies rather than automatically adjusting balances.
- Complete region-4 sandbox journeys for deposit, withdrawal, failed/duplicate callbacks, and recovery by week 4. Validate region-specific configuration and provider identity independently; never copy live credentials between regions. By week 6, complete an integrated rehearsal covering regulatory differences, realistic traffic, batch limits, and rollback/containment.
- Deliver in small increments behind disabled region-specific flags. QA helps define acceptance cases before implementation; engineers supply automated evidence before review. Give the highest-risk incident actions priority over optional roadmap features.

Days 43-70: prove readiness and launch region 4.

- By week 8, rehearse deployment, credential checks, migrations, incident escalation, and recovery with engineers other than S. and T. Prepare Support and the on-call rota. Require no unexplained reconciliation discrepancies outside agreed provider-settlement windows, passing critical money tests, usable alerts, and named Finance/Compliance acceptance.
- Start a controlled production pilot in week 9, subject to approvals, with agreed traffic/value limits and stop conditions. Expand toward the week-10 launch only while those conditions hold. Disable the affected flow on duplicate credits, unexplained financial discrepancies, or failed critical controls; investigate and reconcile before resuming. A code rollback cannot undo money already transferred.
- If readiness slips, escalate early with evidence and options: remove optional launch scope, obtain specific provider support, or acknowledge the date is at risk. I will not describe an unready launch as safe to protect the forecast.

Days 71-90: stabilise and test whether the changes worked.

Review launch outcomes, production defects, recovery drills, and team feedback. Finish high-risk corrective actions, retire steps that provide no useful protection, and agree the next quarter’s priorities. Demonstrate that trained backups can handle wallet and pipeline recovery without their original experts.

Explicitly deferred this quarter: a broad reorganisation, platform/microservice rewrites, a new data platform, wholesale test-suite replacement, and optional promotions or UI improvements competing with the launch. Maintain necessary security, regulatory, and live-service work. Negotiate displaced commitments with Product; do not hide them in an expanding backlog.

Three process changes, owned by the teams

1. **Money-change gate:** a short PR/release check for account authorization, exact money arithmetic, duplicate/concurrent execution, and rollback/reconciliation evidence. A qualified peer reviews; the author owns tests. Automate repeatable checks. This addresses reviewed-but-unsafe changes; success is fewer escaped money defects and demonstrated invariant tests, not more signatures.

2. **Incident learning loop:** one lead, a timestamped incident record, regular stakeholder updates, and a short review within two working days of serious incidents. Select a few corrective actions with owners/dates and check them weekly. Success is quicker containment and recovery, plus fewer repeated failure mechanisms.

3. **Small releases with shared readiness:** QA joins planning; engineers test as they build. Release acceptance covers the affected risks and executable recovery steps. Review launch dependencies and exceptions in an existing weekly planning meeting. Success is less late rework and waiting, with reliable releases and evidenced launch milestones.

For the **two QA**, replace the final testing queue with early risk-based involvement and developer-owned regression tests; do not make two people responsible for everyone’s quality. In week one, work with Recruiting to identify and resolve the reasons the **two backend roles** have remained unfilled for three months. Prioritise payments/reliability experience and backend capability that strengthens CRM/pipeline coverage; adjust the brief or hiring process with Recruiting if evidence warrants it. Reassess after diagnosis. Plan onboarding and mentoring capacity, and do not count unfilled roles as delivered capacity.

Measures and people

By day 14, assemble an initial baseline from deployment records, incidents, and QA work; record gaps rather than invent precision. Review four measures weekly, segmented where region/brand differences matter:

Measure	What decision it supports
Confirmed money incidents and customer/business exposure; unexplained reconciliation items and age	Zero money incidents is the objective. Reconciliation checks whether a quiet incident count reflects correctness or missed detection; investigate overdue exceptions daily.
Detection, containment, and recovery time for serious incidents	Identify which response stage is deteriorating; verify the runbooks and backup responders through drills, not a small-sample average alone.
Change lead time and share of releases causing incidents, rollback, or emergency fixes	Seek a return toward safe weekly releases; speed improving while failures rise is not progress.
Time waiting for QA and late regression rework	Check whether shared testing actually removes the release bottleneck. Review at weeks 4 and 8 and adjust the specific source of delay.

I will meet each lead and a cross-section of engineers/QA in the first week, then hold regular one-to-ones. With **S.**, verify the feedback concern privately, set clear expectations for respectful, actionable reviews, and follow up on observed behaviour. Protect time for pairing and reward others becoming independent; technical importance does not excuse harmful conduct. Repeated issues receive explicit performance follow-through.

Initial allocation of the seven Payments & Platform engineers:

- **Three on money controls:** Fix duplicate-payment protection, money calculations, and wallet/ledger consistency. S. and D. work together in this group.
- **Three on region-4 integration:** Connect and test the new payment providers, led by another experienced engineer.
- **One rotating interrupt owner:** Respond first to production alerts, urgent support requests, and unexpected issues. Rotate the duty; senior backup comes from the same seven engineers.
- **Junior support:** Pair juniors with experienced engineers within these groups for sensitive payment changes.
- **Weekly adjustment:** Reassign capacity based on incidents, risks, and launch progress. Revise scope or timing when needed instead of relying on overtime.
- **Other squads:** CRM & Data and Web & Backoffice retain their region-4 work and existing services.
- **D. and the other mid-level engineers:** Give them specific payment-control or integration tasks to lead. Provide coaching and review experience so more engineers can safely review wallet changes, reducing dependence on S.
- **Junior engineers:** Pair them with experienced colleagues, assign manageable tasks, and encourage them to flag risks or ask questions.
- **T.:** Reduce less urgent CRM work, train a backup, and share pipeline support so T. is not the only person who can handle problems.
- **L.:** Design a practical release process together. Allocate time for mentoring engineers in other squads and limit that workload, rather than expecting L. to absorb it alongside normal duties.

Part B - PR #482

PR review

Request changes - block merge. Resolve the following money-safety risks before release. Snippets below are excerpts from the diff.

1. Brand isolation - blocking

```
const members = await Member.findAll();
```

Issue: `accountId` is unused; running one brand's job can pay other brands.
Required output: Scope members, bets, wallets, and payouts to the authorised account; enforce ownership. A two-brand test must leave the other brand untouched.

2. Duplicate payments - blocking

```
const weekStart = new Date(Date.now() - 7 * 24 * 60 * 60 * 1000);
await CashbackPayout.create({ memberId: member.id, amount: cashback, weekStart });
```

Issue: The period moves on retries; no database uniqueness guard prevents repeat credits.
Required output: Use a fixed scheduled `[start,end)` period and a unique account/member/period entitlement. Retries and concurrent runs must credit once.

3. Partial writes and lost updates - blocking

```
wallet.balance = String(parseFloat(wallet.balance) + cashback);
await wallet.save();
```

Issue: Payout and ledger writes follow separately; failures leave mismatches, and concurrent deposits can be overwritten.
Required output: Use canonical wallet posting: claim, balance, and ledger in one transaction with an atomic increment or established locking. Verify rollback and concurrent deposits.

4. Money precision - blocking

```
netLoss += parseFloat(bet.amount) - parseFloat(bet.payout);
amount: { type: Sequelize.FLOAT, allowNull: false },
```

Issue: Floating-point calculations and storage lose decimal precision.
Required output: Use string-input `dec()`, approved rounding, and `DECIMAL(36,18)` with string model values. Assert exact balances and matching payout/ledger amounts.

5. Authorisation - blocking unless enforced upstream

```
adminRouter.post('/cashback/run', async (req, res, next) => {
```

Issue: Authentication, payout permission, and account access checks are not visible; UUID validation is insufficient.
Required output: Demonstrate upstream enforcement or add checks. Reject unauthorised callers and cross-account requests.

6. Execution and recovery - before enabling

```
console.log('cashback failed for member', member.id, err);
res.json({ ok: true, ...result });
```

Issue: Unbounded HTTP processing hides partial failures and missing wallets.
Required output: Provide bounded batches/concurrency, resumable run IDs, failed/skipped outcomes, reconciliation, and alerts. Validate realistic volume without starving wallet traffic.

Conversation with S. and D.

- **Explain why the merge is paused:** Show S. and D. how running the job twice could pay the same cashback twice, and how a failure after updating the wallet could leave no matching ledger entry. Check whether existing shared code or database controls already prevent these problems.

- **Let D. lead the correction with support:** D. remains responsible for fixing the change, with another engineer helping test failures and simultaneous activity. Transaction tests check that related writes succeed or roll back together. Concurrency tests check that cashback and a simultaneous deposit do not overwrite each other. Keep feedback specific and respectful.

- **Discuss S.'s approval privately:** Understand why S. approved the code: deadline pressure, an oversight, or unsafe patterns already used elsewhere. Agree that a staff engineer's review must check brand separation, exact money calculations, repeated requests, and failure recovery. D. still owns the code, but should not carry all the blame when a senior reviewer approved it.

- **Make safe implementation easier for the squad:** Provide shared, reviewed wallet functions that safely update balances and record transactions. Add tests against a real test database proving that one entitlement produces one credit and wallet/ledger changes stay consistent. Use a short money-safety review checklist. Train another wallet reviewer to reduce dependence on S. Periodically check reviewed changes and their results so the Engineering Manager oversees quality without personally catching every defect.

Part C1 - Financial incident action plan

Objective: Contain incorrect credits, establish exposure, and restore safely. Assume Finance/operations coverage.

1. At 01:40 - activate the response

- Declare a serious financial incident; open an incident channel and decision log.
- Assign coordination/CEO updates to the Engineering Manager, technical investigation to S., evidence recording to an available engineer, customer reports to Support, and reconciliation to Finance. Avoid dependence on T.

2. First 15 minutes - contain potential losses

- Recommend and execute a pause of affected region-2 crediting within incident authority; obtain urgent approval where needed. See email example below.
- Assess withdrawal restrictions for potentially overcredited wallets. Expand containment if safe isolation fails.
- Preserve transaction identifiers, logs, and deployment history without delaying containment.

To: CEO; **CC:** Support, Finance, on-call team
Subject: Region 2 - temporary crediting pause
Status: Incorrect balances reported; financial loss unconfirmed.
Recommendation: Pause affected crediting pending verification; assess affected withdrawals.
Updates: Every 15 minutes during the active phase, including no-change updates; communicate material developments immediately.

3. First hour - establish the facts

- Inspect recent changes, duplicate callbacks, and ledger consistency.
- Reconcile wallets, ledger, provider records, and withdrawals; distinguish inflated balances from withdrawn funds.
- Report confirmed exposure and uncertainties; coordinate Support messaging. Agree a reduced update frequency only after containment.

4. Recovery through hour 24 - restore and hand over

- Require a reviewed fix, failure/retry tests, reconciliation evidence, and monitored limited resumption with stop conditions.
- Apply approved, auditable financial corrections; never delete ledger entries. Do not assume code rollback reverses transfers.
- Arrange rested handover, customer/regulatory communications through responsible owners, and an incident review with corrective-action owners and deadlines.

Part C2 - Release-checklist action plan

Objective: Resolve the delivery concern, retain essential financial controls, and establish one agreed release process.

1. First 60 seconds - restore meeting focus

- Acknowledge the concern calmly; invite one example of a check causing delay.
- Recognise valid criticism; avoid a public argument or a vote on personal authority.
- Confirm that current safeguards remain mandatory until an alternative is approved.
- Assign a same-day follow-up, state when the squads will receive a decision, and resume planning.

2. Same day - establish facts and expectations

- Meet L. privately; examine specific delays, workload, duplicated checks, and gaps in consultation.
- Accept responsibility for unnecessary process or an unclear rollout.
- Protect open challenge while clarifying that encouraging unilateral bypass creates conflicting instructions and financial risk.

3. Before the next release - agree a better approach

- Involve L., an affected engineer, and QA in assessing each disputed check.
- Link each check to a concrete failure risk. Remove duplication, automate repeatable checks, and preserve evidence for duplicate protection, money accuracy, account isolation, and recovery.
- Approve a shorter checklist for a two-release trial. Publish the rationale, owners, required evidence, and exception procedure to both squads.
- Require named approval and documented compensating controls for emergency exceptions; prohibit informal opt-outs.

4. After two releases - verify and follow through

- Compare QA waiting, change lead time, and escaped defects against the baseline. Review results with L.; retain effective changes and revise ineffective ones.
- Separate disagreement from misconduct. Address repeated encouragement to bypass agreed controls through private coaching, clear expectations, and proportionate performance follow-through if necessary.

AI disclosure: OpenAI (ChatGPT) was used to help interpret the scenario and suggest and discuss priorities before drafting this document.